Huawei AI Certification Training

AI Final Exam

Lab Guide

ISSUE:1.0

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2021. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions

HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Huawei Technologies Co., Ltd.

Address: Huawei Industrial Base Bantian, Longgang Shenzhen 518129 People's Republic of China

Website: http://e.huawei.com

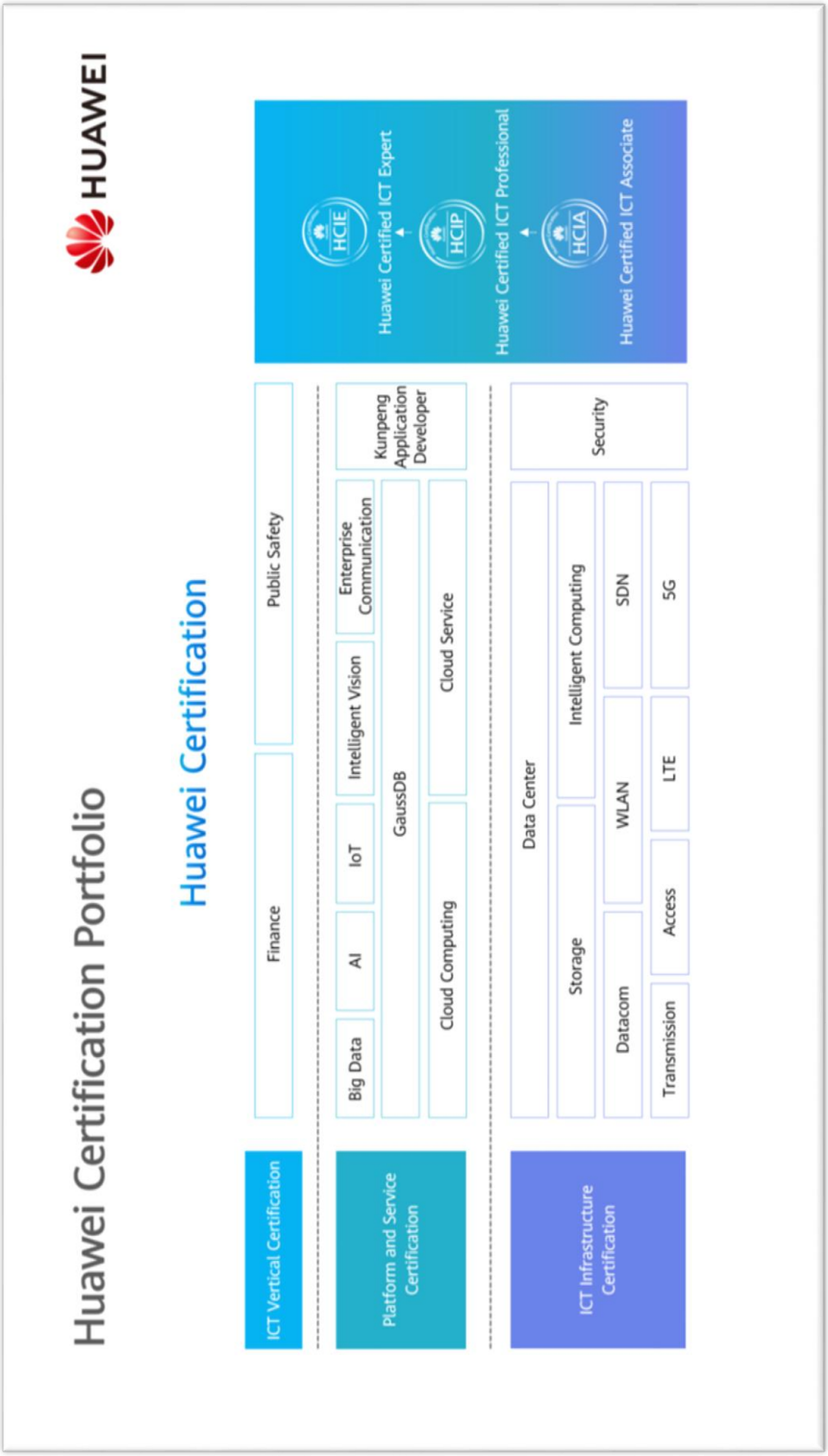
Huawei Certificate System

Huawei Certification follows the "platform + ecosystem" development strategy, which is a new collaborative architecture of ICT infrastructure based on "Cloud-Pipe-Terminal". Huawei has set up a complete certification system consisting of three categories: ICT infrastructure certification, Platform and Service certification and ICT vertical certification, and grants Huawei certification the only all-range technical certification in the industry.

Huawei offers three levels of certification: Huawei Certified ICT Associate (HCIA), Huawei Certified ICT Professional (HCIP), and Huawei Certified ICT Expert (HCIE).

HCIA-AI V3.0 aims to train and certify engineers who are capable of designing and developing AI products and solutions using algorithms such as machine learning and deep learning.

HCIA-AI V3.0 certification demonstrates that: You know the development history of AI, Huawei Ascend AI system and full-stack all-scenario AI strategies, and master traditional machine learning and deep learning algorithms; you can use the TensorFlow and MindSpore development frameworks to build, train, and deploy neural networks; you are competent for sales, marketing, product manager, project management, and technical support positions in the AI field.


Huawei Certification Portfolio
HUAWEI
Huawei Certification
ICT Vertical Certification
Finance
Public Safety
Platform and Service Certification
Big Data
AI
IoT
Intelligent Vision
Enterprise Communication
Kunpeng Application Developer
GaussDB
Cloud Computing
Cloud Service
ICT Infrastructure Certification
Data Center
Security
Storage
Intelligent Computing
Datacom
WLAN
SDN
Transmission
Access
LTE
5G
HCIE
Huawei Certified ICT Expert
HCIP
Huawei Certified ICT Professional
HCIA
Huawei Certified ICT Associate

Contents

1 Handwritten Digit Recognition....4
1.1 Introduction....4
1.1.1 About This Lab....4
1.1.2 Objectives....4
1.2 Procedure....4
1.2.1 Project Description and Dataset Acquisition....4
1.2.2 Dataset Preprocessing and Visualization....6
1.2.3 DNN Construction....7
1.2.4 CNN Construction....9
1.2.5 Prediction Result Visualization....10

1 Handwritten Digit Recognition

1.1 Introduction

1.1.1 About This Lab

Handwritten digit recognition is a common image recognition task where computers recognize text in handwriting images. Different from printed fonts, handwriting of different people has different sizes and styles, making it difficult for computers to recognize handwriting. This project applies deep learning and TensorFlow tools to train and build models based on the MNIST handwriting dataset. This chapter describes the basic process of TensorFlow computing and basic elements for building a network.

1.1.2 Objectives

Upon completion of this experiment, you will be able to:

- Master the basic process of TensorFlow computing.
- Be familiar with the basic elements of network building, including dataset, network model building, model training, and model validation.

1.2 Procedure

This experiment involves the following steps:

- Reading the MNIST handwritten digit dataset.
- Getting started with TensorFlow by using simple mathematical models.
- Implementing softmax regression by using high-level APIs.
- Building a multi-layer CNN.
- Implementing a CNN by using high-level APIs.
- Visualizing prediction results.

1.2.1 Project Description and Dataset Acquisition

1.2.1.1 Project Description

Handwritten digit recognition is a common image recognition task where computers recognize text in handwriting images. Different from printed fonts, handwriting of different people has different sizes and styles, making it difficult for computers to recognize handwriting. This project applies deep learning and TensorFlow tools to train and build models based on the MNIST handwriting dataset.

1.2.1.2 Data Acquisition and Processing

1.2.1.2.1 Dataset

1. The MNIST dataset is from the National Institute of Standards and Technology (NIST).
2. The dataset consists of handwritten digits from 250 individuals, of which 50% are high school students and 50% are staff from Bureau of the Census.
3. You can download the dataset from http://yann.lecun.com/exdb/mnist/, which includes the following four parts:

   - Training set images: train-images-idx3-ubyte.gz (9.9 MB, 47 MB after decompression, including 60,000 samples)

   - Training set labels: train-labels-idx1-ubyte.gz (29 KB, 60 KB after decompression, including 60,000

   labels)

   - Test set images: t10k-images-idx3-ubyte.gz (1.6 MB, 7.8 MB after decompression, including 10,000

   samples)

   - Test set labels: t10k-labels-idx1-ubyte.gz (5 KB, 10 KB after decompression, including 10,000 labels)
4. The MNIST dataset is an entry-level computer vision dataset that contains images of various handwritten digits.

   It also contains one label for each image, to clarify the correct digit. For example, the labels for the preceding four images are 5, 0, 4, and 1.

1.2.1.2.2 MNIST Dataset Reading

Download the MNIST dataset directly from the official TensorFlow website and decompress it.

Code:

```
import os
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, optimizers, datasets
from matplotlib import pyplot as plt
import numpy as np
(x_train_raw, y_train_raw), (x_test_raw, y_test_raw) = datasets.mnist.load_data()
print(y_train_raw[0])
print(x_train_raw.shape, y_train_raw.shape)
print(x_test_raw.shape, y_test_raw.shape)
#Convert the labels into onehot code.
num_classes = 10
y_train = keras.utils.to_categorical(y_train_raw, num_classes)
y_test = keras.utils.to_categorical(y_test_raw, num_classes)
print(y_train[0])
```

Output:

```
5
```

```
(60000, 28, 28) (60000,)
(10000, 28, 28) (10000,)
[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

In the MNIST dataset, the images are a tensor in the shape of [60000, 28, 28]. The first dimension is used to extract images, and the second and third dimensions are used to extract pixels in each image. Each element in this tensor indicates the strength of a pixel in an image. The value ranges between 0 and 255.

Label data is one-hot vectors. In a one-hot vector, one digit is 1, and digits in other dimensions are all 0s. For example, label 1 may be represented as [0,1,0,0,0,0,0,0,0,0,0,0]. Therefore, the labels are a digital matrix of [60000, 10].

1.2.2 Dataset Preprocessing and Visualization

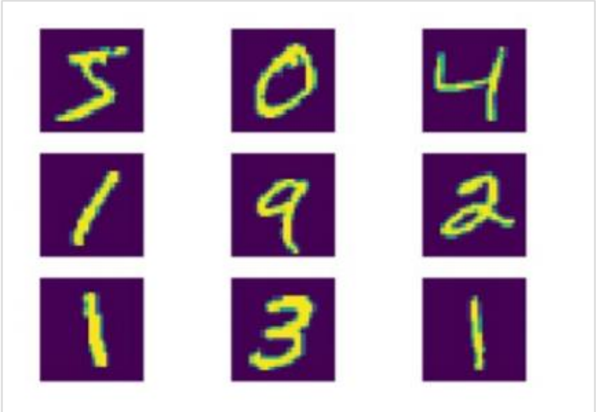
1.2.2.1 Data Visualization

Draw the first nine images.

Code:

```
plt.figure()
for i in range(9):
  plt.subplot(3,3,i+1)
  plt.imshow(x_train_raw[i])
  #plt.ylabel(y[i].numpy())
  plt.axis('off')
plt.show()
```

Output:



Data processing is required because when we build a fully connected network, the output should be in the form of vectors instead of an image matrix. Therefore, you need to sort the images into vectors.

Code:

```
#Convert a 28 x 28 image into a 784 x 1 vector.
x_train = x_train_raw.reshape(60000, 784)
```

```
x_test = x_test_raw.reshape(10000, 784)
```

Currently, the dynamic range of pixels is 0 to 255. Image pixels are usually normalized to the range of 0 to 1 during processing of image pixel values.

Code:

```
#Normalize image pixel values.
x_train = x_train.astype('float32')/255
x_test = x_test.astype('float32')/255
```

1.2.3 DNN Construction

1.2.3.1 Building a DNN Model

Code:

```
#Create a deep neural network (DNN) model that consists of three fully connected layers and two
RELU activation functions.
model = keras.Sequential([
  layers.Dense(512, activation='relu', input_dim = 784),
  layers.Dense(256, activation='relu'),
  layers.Dense(124, activation='relu'),
layers.Dense(num_classes, activation='softmax')])
model.summary()
```

Output:

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 512)               401920
_________________________________________________________________
dense_1 (Dense)              (None, 256)               131328
_________________________________________________________________
dense_2 (Dense)              (None, 124)               31868
_________________________________________________________________
dense_3 (Dense)              (None, 10)                1250
=================================================================
Total params: 566,366
Trainable params: 566,366
Non-trainable params: 0
_________________________________________________________________
```

layer.Dense() indicates a fully connected layer, and **activation** indicates a used activation function.

1.2.3.2 Compiling the DNN Model

Code:

```
Optimizer = optimizers.Adam(0.001)
model.compile(loss=keras.losses.categorical_crossentropy,
  optimizer=Optimizer,
```

```
metrics=['accuracy'])
```

In the preceding example, the loss function of the model is cross entropy, and the optimization algorithm is the Adam gradient descent method.

1.2.3.3 Training the DNN Model

Code:

```
#Fit the training data to the model by using the fit method.
model.fit(x_train, y_train,
 batch_size=128,
 epochs=10,
 verbose=1)
```

Output:

```
Epoch 1/10
60000/60000 [==============================] - 7s 114us/sample - loss: 0.2281 - acc: 0.9327s -
loss: 0.2594 - acc: 0. - ETA: 1s - loss: 0.2535 - acc: 0.9 - ETA: 1s - loss:
Epoch 2/10
60000/60000 [==============================] - 8s 129us/sample - loss: 0.0830 - acc: 0.9745s -
loss: 0.0814 - ac
Epoch 3/10
60000/60000 [==============================] - 8s 127us/sample - loss: 0.0553 - acc: 0.9822
Epoch 4/10
60000/60000 [==============================] - 7s 117us/sample - loss: 0.0397 - acc: 0.9874s -
los
Epoch 5/10
60000/60000 [==============================] - 8s 129us/sample - loss: 0.0286 - acc: 0.9914
Epoch 6/10
60000/60000 [==============================] - 8s 136us/sample - loss: 0.0252 - acc: 0.9919
Epoch 7/10
60000/60000 [==============================] - 8s 129us/sample - loss: 0.0204 - acc: 0.9931s -
lo
Epoch 8/10
60000/60000 [==============================] - 8s 135us/sample - loss: 0.0194 - acc: 0.9938
Epoch 9/10
60000/60000 [==============================] - 7s 109us/sample - loss: 0.0162 - acc: 0.9948
Epoch 10/10
60000/60000 [==============================] - ETA: 0s - loss: 0.0149 - acc: 0.994 - 7s
117us/sample - loss: 0.0148 - acc: 0.9948
```

Epoch indicates a specific round of training. In the preceding example, full data is iterated for 10 times.

1.2.3.4 Evaluating the DNN Model

Code:

```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

Output:

```
Test loss: 0.48341113169193267
Test accuracy: 0.8765
```

The evaluation shows that the model accuracy reaches 0.87, and 10 training iterations have been performed.

1.2.3.5 Saving the DNN Model

Code:

```
model.save('./mnist_model/final_DNN_model.h5')
```

1.2.4 CNN Construction

The conventional CNN construction method helps you better understand the internal network structure but has a large code volume. Therefore, attempts to construct a CNN by using high-level APIs are made to simplify the network construction process.

1.2.4.1 Building a CNN Model

Code:

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
model=keras.Sequential() #Create a network sequence.
##Add the first convolutional layer and pooling layer.
model.add(keras.layers.Conv2D(filters=32,kernel_size = 5,strides = (1,1),
  padding = 'same',activation = tf.nn.relu,input_shape = (28,28,1)))
model.add(keras.layers.MaxPool2D(pool_size=(2,2), strides = (2,2), padding = 'valid'))
##Add the second convolutional layer and pooling layer.
model.add(keras.layers.Conv2D(filters=64,kernel_size = 3,strides = (1,1),padding = 'same',activation =
tf.nn.relu))
model.add(keras.layers.MaxPool2D(pool_size=(2,2), strides = (2,2), padding = 'valid'))
##Add a dropout layer to reduce overfitting.
model.add(keras.layers.Dropout(0.25))
model.add(keras.layers.Flatten())
##Add two fully connected layers.
model.add(keras.layers.Dense(units=128,activation = tf.nn.relu))
model.add(keras.layers.Dropout(0.5))
model.add(keras.layers.Dense(units=10,activation = tf.nn.softmax))
```

In the preceding network, two convolutional pooling layers are added by using **keras.layers**, and then a dropout layer is added to prevent overfitting. Finally, two fully connected layers are added.

1.2.4.2 Compiling and Training the CNN Model

Code:

```
#Expand data dimensions to adapt to the CNN model.
X_train=x_train.reshape(60000,28,28,1)
X_test=x_test.reshape(10000,28,28,1)
model.compile(optimizer='Adam',loss="categorical_crossentropy",metrics=['accuracy'])
model.fit(x=X_train,y=y_train,epochs=5,batch_size=128)
```

Output:

```
Epoch 1/5
55000/55000 [==============================] - 49s 899us/sample - loss: 0.2107 - acc: 0.9348
Epoch 2/5
55000/55000 [==============================] - 48s 877us/sample - loss: 0.0793 - acc: 0.9763
Epoch 3/5
55000/55000 [==============================] - 52s 938us/sample - loss: 0.0617 - acc: 0.9815
Epoch 4/5
55000/55000 [==============================] - 48s 867us/sample - loss: 0.0501 - acc: 0.9846
Epoch 5/5
55000/55000 [==============================] - 50s 901us/sample - loss: 0.0452 - acc: 0.9862
<tensorflow.python.keras.callbacks.History at 0x214bbf34ac8>
```

During training, the network training data is iterated for only five times. You can increase the number of network iterations to check the effect.

1.2.4.3 Verifying the CNN Model

Code:

```
test_loss,test_acc=model.evaluate(x=X_test,y=y_test)
print("Test Accuracy %.2f"%test_acc)
```

Output:

```
10000/10000 [==============================] - 2s 185us/sample - loss: 0.0239 - acc: 0.9921
Test Accuracy 0.99
```

The verification shows that accuracy of the CNN model reaches up to 99%.

1.2.4.4 Saving the CNN Model

Code:

```
test_loss,test_acc=model.evaluate(x=X_test,y=y_test)
model.save('./mnist_model/final_CNN_model.h5')
print("Test Accuracy %.2f"%test_acc)
```

Output:

```
10000/10000 [==============================] - 5s 489us/sample - loss: 0.0263 - acc: 0.9920s -
loss: 0.0273 - ac
Test Accuracy 0.99
```

1.2.5 Prediction Result Visualization

1.2.5.1 Loading the CNN Model

Code:

```
from tensorflow.keras.models import load_model
new_model = load_model('./mnist_model/final_CNN_model.h5')
new_model.summary()
```

Output:

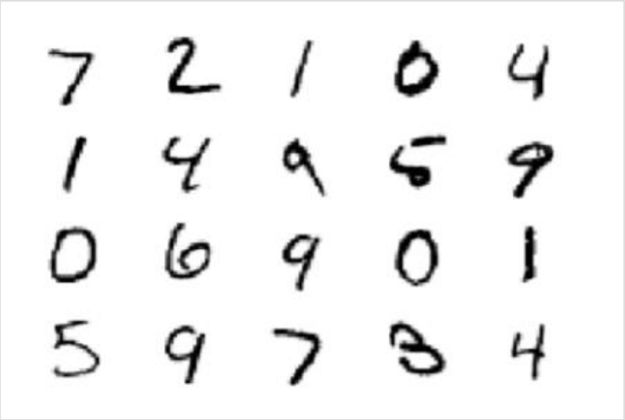
```
_________________________________________________________________
Layer (type) Output Shape Param #
=================================================================
conv2d (Conv2D) (None, 28, 28, 32) 832
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 14, 14, 32) 0
_________________________________________________________________
conv2d_1 (Conv2D) (None, 14, 14, 64) 18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 7, 7, 64) 0
_________________________________________________________________
dropout (Dropout) (None, 7, 7, 64) 0
_________________________________________________________________
flatten (Flatten) (None, 3136) 0
_________________________________________________________________
dense_4 (Dense) (None, 128) 401536
_________________________________________________________________
dropout_1 (Dropout) (None, 128) 0
_________________________________________________________________
dense_5 (Dense) (None, 10) 1290
=================================================================
Total params: 422,154
Trainable params: 422,154
Non-trainable params: 0
_________________________________________________________________
```

Visualize prediction results.

Code:

```
#Visualize test set output results.
import matplotlib.pyplot as plt
%matplotlib inline
def res_Visual(n):
 final_opt_a=new_model.predict_classes(X_test[0:n])#Perform predictions on the test set by using the
model.
 fig, ax = plt.subplots(nrows=int(n/5),ncols=5 )
 ax = ax.flatten()
 print('prediction results of the first {} images:'.format(n).)
 for i in range(n):
 print(final_opt_a[i],end=',')
 if int((i+1)%5) ==0:
 print('\t')
 #Visualize image display.
 The img = X_test[i].reshape((28,28))#Read each row of data in the format of n-dimensional array
(ndarray).
 plt.axis("off")
 ax[i].imshow(img, cmap='Greys', interpolation='nearest')#Visualization
 ax[i].axis("off")
 print('first {} images in the test set:'.format(n)'.)
res_Visual(20)
```

Output:

```
Prediction results of the first 20 images:
7,2,1,0,4,
```

1,4,9,5,9,
0,6,9,0,1,
5,9,7,3,4,
First 20 images of the test set: